

M4_Practice

Danni Martin

2024-11-18

Avocado Prices

Chilango's Kitchen specializes in tacos and burritos that are made to order in front of the customer. Guacamole is the perfect pairing to their delicious food and one of the restaurant's best sellers. Their guacamole uses just six ingredients: avocados, lime juice, cilantro, red onion, jalapeño, and kosher salt. Because of its popularity, each restaurant goes through approximately five cases of avocados daily, amounting to more than 44,000 pounds annually. Chilango's Kitchen wants you to provide insights on the price of avocados in California.

The Hass Avocado Board [Links to an external site.](#) provides the data set which contains weekly retail scan data of U.S. retail volume (units) and price. Retail scan data comes directly from retailers' cash registers based on actual retail sales of Hass avocados. The data reflects an expanded, multi-outlet reporting aggregating the following channels: grocery, mass, club, drug, dollar, and military. The average price of avocados in the table reflects a per unit (per avocado) cost, even when multiple units (avocados) are sold in bags. Other avocados (e.g., greenskins) are not included in this data.

Note

In general, the data is recorded weekly. However, there is an entry on 01/01/2018 that is right after 12/31/2017. This is a single observation that is not weekly. There are also missing dates from 12/02/2018–12/31/2018.

Inspecting the Data

```
#Load tidyverse
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.1
v ggplot2     3.5.1     v tibble     3.2.1
v lubridate  1.9.3     v tidyr      1.3.1
v purrr       1.0.2

-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
#Load data
avocado<-read_csv("https://jagelves.github.io/Data/avocado2020.csv")
```

```
Rows: 33045 Columns: 13
-- Column specification -----
Delimiter: ","
chr  (3): date, type, geography
dbl (10): average_price, total_volume, 4046, 4225, 4770, total_bags, small_b...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
#Preview data
spec(avocado)
```

```
cols(
  date = col_character(),
  average_price = col_double(),
  total_volume = col_double(),
  `4046` = col_double(),
  `4225` = col_double(),
  `4770` = col_double(),
  total_bags = col_double(),
  small_bags = col_double(),
  large_bags = col_double(),
  xlarge_bags = col_double(),
  type = col_character(),
  year = col_double(),
  geography = col_character()
)
```

Lubridate Package

Designed to make working with date and time easier and more intuitive. Part of tidyverse.

```
#Use lubridate to parse/coerce dates
library(lubridate)
avocado$date <- mdy(avocado$date)

class(avocado$date)
```

```
[1] "Date"
```

Piping and dplyr

dplyr is commonly used with “piping.” Generally speaking, “piping” allows us to chain functions. “Piping” (%>% or |>) passes the object on the left of the pipe as the first argument in the function to the right of the pipe. We can illustrate this using the select() and arrange() functions.

```
#Select() - select variables.
#Arrange() - sorts data in ascending order.
#Desc() - sorts data in descending order.

avocado %>% select(c(average_price,geography)) %>%
  arrange(desc(average_price)) %>% head(5)
```

```
# A tibble: 5 x 2
  average_price geography
      <dbl> <chr>
1         3.25 San Francisco
2         3.17 Tampa
3         3.12 San Francisco
4         3.05 Miami/Ft. Lauderdale
5         3.04 Raleigh/Greensboro
```

```
#Filter out record that is not weekly. filter() chooses rows that meet a certain condition.
#Reassign filtered data to avocado.
avocado %>% filter(date!=ymd("2018-01-01")) -> avocado
```

```
#California's average price for organic avocados between 2015-2018
avocado %>%
  filter(geography=="California", type=="organic",
         year<=2018) %>%
  select(date, average_price, geography) -> cali
```

```
#Standardize time intervals are important for timeseries.
#Confirm this using pull() and diff() function. Pull() vectorizes date variable, diff() returns
cali %>% pull(date) %>% diff()
```

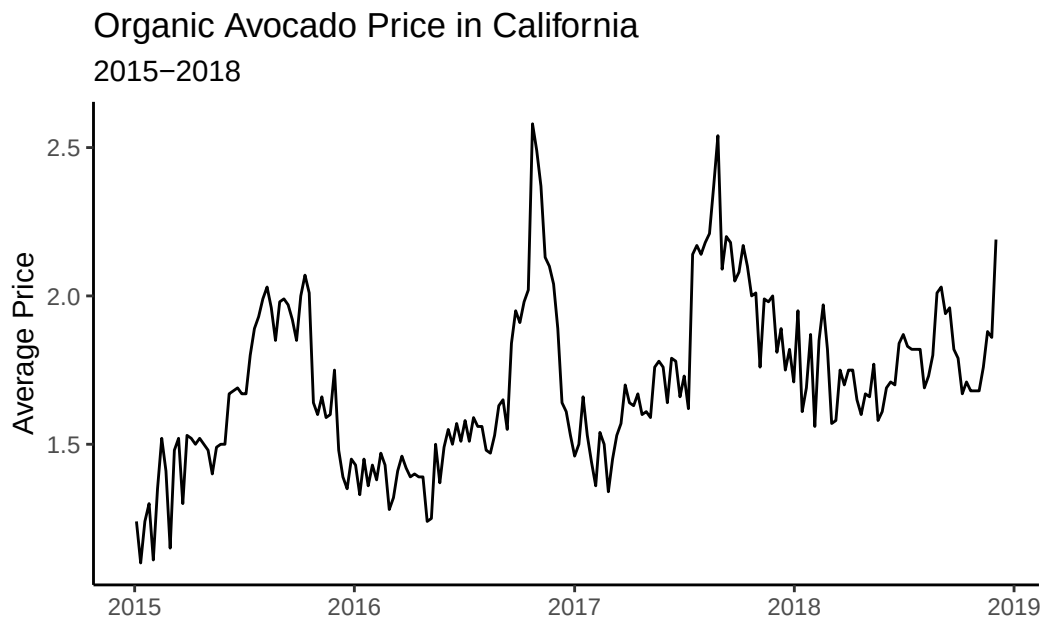
Time differences in days

```
[1] 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
[38] 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
[75] 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
[112] 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
[149] 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
[186] 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
```

Visualizing the data.

To visualize the data, we will be using ggplot2. One of the main functions in ggplot2 is the `aes()` function. This function sets the plotting canvas and determines the mapping of variables. The `geom_line()` function specifies the type of plot. In time series, we will use the line plot regularly. Labels for the graph are easily set with the `labs()` function and there are plenty of themes available to customize your visualization. Below, the `theme_classic()` is displayed.

```
ggplot(data=cali) +
  geom_line(mapping=aes(x=date,y=average_price, group=geography),color="black") +
  theme_classic() +
  labs(x="",
       y="Average Price",
       title="Organic Avocado Price in California",
       subtitle="2015-2018")
```



The average price of avocados in California increased during the period considered. It reached a maximum of about 2.60 in 2017 and a minimum of 1.10 in 2015. There is also a seasonal pattern with low prices at the beginning of the year and peaks mid-year. `### tsibbles` are defined by a time index (i.e., the date) that has a standard interval (i.e., days, weeks, months, and years) and keys (i.e., dimensions or observational units).

```
library(fpp3)
```

```
Warning: package 'fpp3' was built under R version 4.4.2
```

```
Registered S3 method overwritten by 'tsibble':
```

```
  method      from
as_tibble.grouped_df dplyr
```

```
-- Attaching packages ----- fpp3 1.0.1 --
```

```
v tsibble      1.1.5      v feasts      0.4.1
v tsibbledata 0.4.1      v fable       0.4.1
```

```
Warning: package 'tsibble' was built under R version 4.4.2
```

```
Warning: package 'tsibbledata' was built under R version 4.4.2
```

Warning: package 'feasts' was built under R version 4.4.2

Warning: package 'fabletools' was built under R version 4.4.2

Warning: package 'fable' was built under R version 4.4.2

```
-- Conflicts ----- fpp3_conflicts --
x lubridate::date()      masks base::date()
x dplyr::filter()        masks stats::filter()
x tsibble::intersect()   masks base::intersect()
x tsibble::interval()    masks lubridate::interval()
x dplyr::lag()            masks stats::lag()
x tsibble::setdiff()     masks base::setdiff()
x tsibble::union()       masks base::union()
```

```
avocado %>%
  as_tsibble(key=c(type, geography),
             index=date, regular=T) %>%
  filter_index("2015-01-04"~"2018-12-02")-> avocadots
```

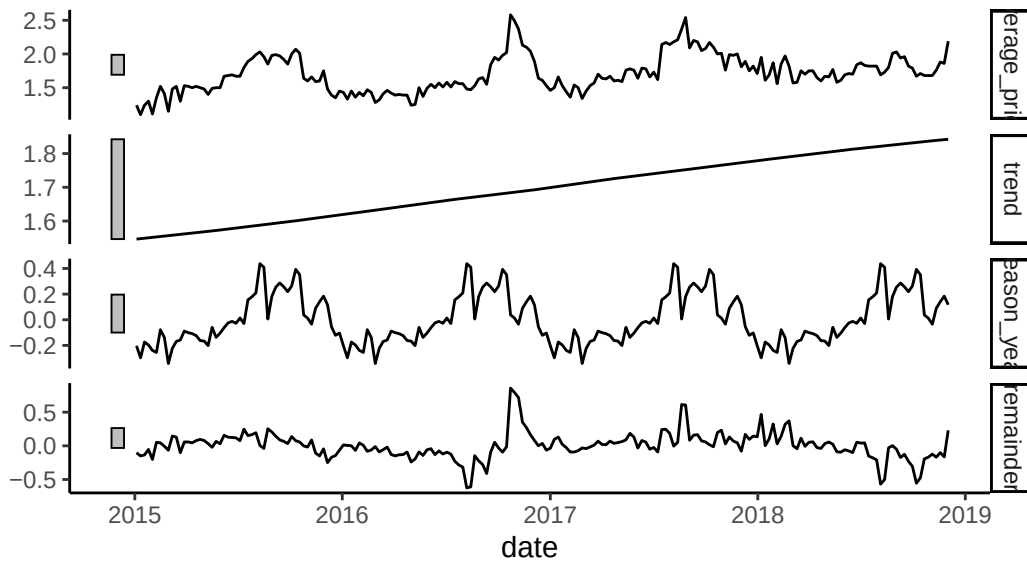
```
#Filter for California
avocadots %>% filter(geography=="California", type=="organic") %>%
  select(date,geography,average_price) -> calits
```

Time Series Decomposition

```
# Season trend decomposition using LOESS (STL)
calits %>%
  model(STL(average_price~trend(window=199)+
           season(window=51), robust=TRUE)) %>%
  components() %>% autoplot()+ theme_classic()
```

STL decomposition

average_price = trend + season_year + remainder



Practice Exercises

Question 1

Strawberry Price data. Graph the time series using ggplot2 and conduct time series decomposition using the fpp3 package.

```
# Import libraries.
library(tidyverse)
library(fpp3)

# Load data.
SB<-read_csv("https://jagelves.github.io/Data/StrawberryPrice.csv")
```

Rows: 524 Columns: 2

-- Column specification -----

Delimiter: ","

chr (2): DATE, Price

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

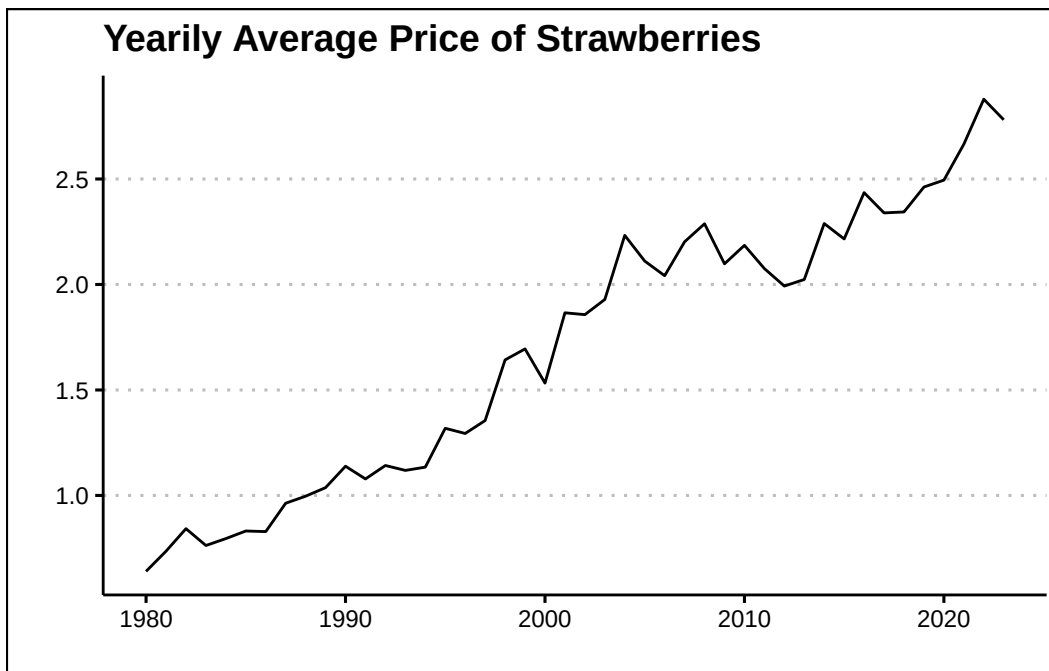
```
# Create tsibble.
SB %>% mutate(Year=year(mdy(DATE))) %>% #Coerce date variable and extract year.
  mutate(Price=as.double(Price)) %>% #Coerce Price to double.
  group_by(Year) %>% #Group by year.
  summarise(APrice=mean(Price, na.rm=T)) %>% #Summarize by average.
  tsibble(index=Year)-> SB_ts
```

Warning: There was 1 warning in `mutate()`.
 i In argument: `Price = as.double(Price)`.
 Caused by warning:
 ! NAs introduced by coercion

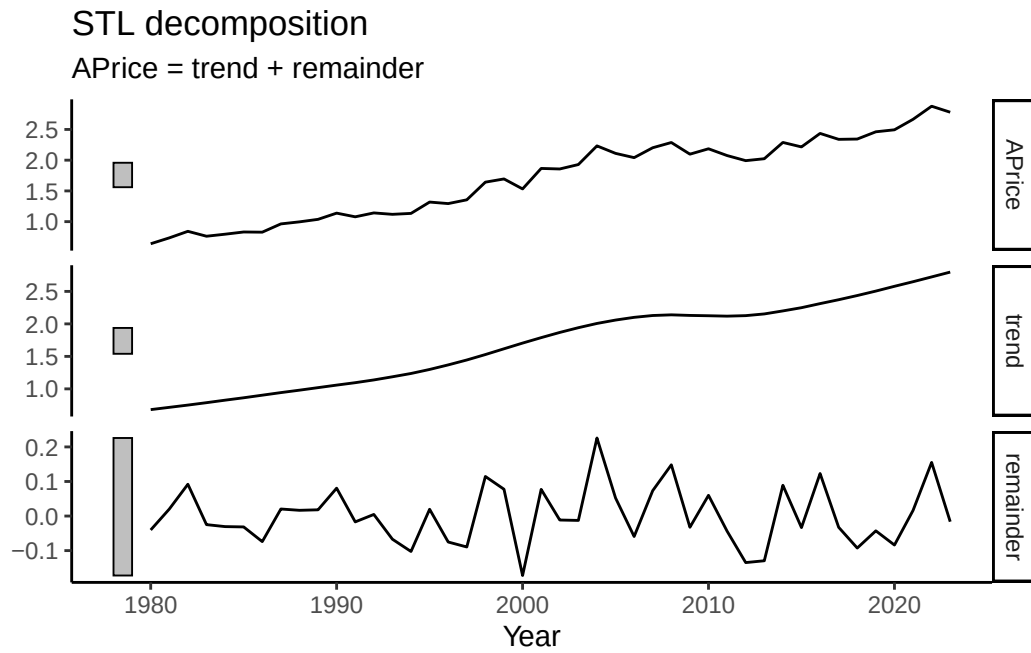
```
# Graph using autoplot() command.
library(ggthemes)
```

Warning: package 'ggthemes' was built under R version 4.4.2

```
SB_ts %>% autoplot(.vars=APrice) +
  theme_clean() +
  labs(title="Yearly Average Price of Strawberries",
       y="", x="")
```



```
#Decomposition
SB_ts %>% model(STL=STL(APrice ~ trend() + season(),
                        robust=T)) %>%
  components() %>% autoplot() + theme_classic()
```



Question 2